

Python QB

Question at Remember Level (2 Marks)

- 1) Define Python Programming Language.
- 2) Enlist Applications for Python Programming.
- 3) List any four editors used for Python Programming.
- 4) Enlist features of Python Programming
- 5) State Limitations of Python Programming.
- 6) Define Indentation in Python Programming.
- 7) How to comment specific lines in Python Programming.
- 8) Define Identifier in Python Programming.
- 9) Enlist ways to run python script.
- 10) Define variables, keywords in Python Programming.

ChatGPT said:

1) Define Python Programming Language.

Python is a high-level, interpreted, and dynamically-typed programming language known for its simplicity and readability. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

2) Enlist Applications for Python Programming.

- Web Development (Django, Flask)
- Data Science & Machine Learning (Pandas, NumPy, Scikit-learn)
- Game Development (Pygame)
- Automation & Scripting
- Cybersecurity & Ethical Hacking
- Embedded Systems (MicroPython)

3) List any four editors used for Python Programming.

- PyCharm
- VS Code
- Jupyter Notebook
- Spyder

4) Enlist features of Python Programming.

- **Easy to Learn & Readable**
- **Interpreted Language** (No compilation needed)
- **Dynamically Typed** (No need to declare variable types)
- **Extensive Libraries & Frameworks**
- **Cross-platform Compatibility**
- **Supports Object-Oriented & Functional Programming**

5) State Limitations of Python Programming.

Python QB

- **Slower Execution Speed** (Because it's interpreted)
- **High Memory Usage** (Not suitable for low-memory devices)
- **Weak in Mobile Development**
- **Runtime Errors** (Due to dynamic typing)

6) Define Indentation in Python Programming.

Indentation in Python refers to the use of spaces or tabs to define blocks of code. It is mandatory and replaces traditional braces {} used in other languages.

Example

```
if True:
    print("Hello, World!") # Indented block
```

7) How to comment specific lines in Python Programming?

Python supports both **single-line** and **multi-line** comments:

- **Single-line comment:**

```
# This is a single-line comment
print("Hello, Python!")
```

- **Multi-line comment (Using triple quotes or multiple #):**

```
"""
This is a
multi-line comment
"""
print("Python is fun!")
```

8) Define Identifier in Python Programming.

An **identifier** is the name given to variables, functions, classes, and objects. It must start with a letter (A-Z or a-z) or an underscore (_) followed by letters, digits (0-9), or underscores.

Example:

```
variable_name = 10 # Valid identifier
_age = 25          # Valid identifier
2name = "John"     # Invalid (Cannot start with a number)
```

9) Enlist ways to run a Python script.

- **Using the Command Line:**

Python QB

```
python script.py
```

- **Using an IDE (e.g., PyCharm, VS Code)**
- **Using Jupyter Notebook**
- **Using Online Python Interpreters (e.g., Replit, Google Colab)**

10) Define Variables and Keywords in Python Programming.

- **Variable:** A variable is a named storage location used to hold values in Python. Variables do not require explicit declaration.

```
x = 10  
name = "Python"
```

- **Keyword:** Keywords are reserved words in Python that have predefined meanings and cannot be used as identifiers.
Example: if, else, while, def, class, etc.

```
import keyword  
print(keyword.kwlist) # Lists all Python keywords
```

State and explain features of Python Programming (Any four). 2) Differentiate Between Interactive mode and Script mode of Python. In tabular Programming. 3) Explain history of Python Programming Language. 4) Write the rules and conventions for declaring a variables. 5) How to run python script explain in detail. 6) Explain structure of python program . 7) Explain commenting in Python Programming. 8) Explain input and output in Python Programming

1) State and Explain Features of Python Programming (Any Four).

Python has several features that make it a powerful programming language:

- **Easy to Learn & Readable:** Python has a simple and readable syntax, making it beginner-friendly.
- **Interpreted Language:** Python code is executed line by line, eliminating the need for compilation.
- **Dynamically Typed:** There is no need to declare variable types explicitly.
- **Extensive Libraries & Frameworks:** Python provides a vast collection of libraries like NumPy, Pandas, TensorFlow, and Django for various applications.

2) Differentiate Between Interactive Mode and Script Mode of Python

Python QB

Feature	Interactive Mode	Script Mode
Execution Type	Executes commands line by line	Executes an entire script file at once
Usage	Used for quick testing and debugging	Used for writing full programs
Output	Immediate output is shown	Output is shown after running the script
Example	>>> print("Hello")	script.py containing print("Hello")

3) Explain History of Python Programming Language.

- Python was created by **Guido van Rossum** in the **late 1980s** and officially released in **1991**.
- It was developed as a successor to the **ABC language** and aimed to be more readable and efficient.
- Python 2 was released in **2000**, followed by Python 3 in **2008**, which introduced several improvements.
- Today, Python is widely used in web development, AI, machine learning, and more.

4) Write the Rules and Conventions for Declaring Variables.

- **Variable names must start with a letter (A-Z, a-z) or an underscore (_).**
- **The name can contain letters, digits (0-9), and underscores (_).**
- **Case-sensitive:** name and Name are different variables.
- **Cannot be a Python keyword:** E.g., if = 10 is invalid.

Example:

```
my_variable = 10 # Valid
_name = "Python" # Valid
2name = "Error" # Invalid (Cannot start with a number)
```

5) How to Run a Python Script?

There are multiple ways to run a Python script:

1. Using the Command Line:

```
python script.py
```

2. Using an IDE (e.g., PyCharm, VS Code):

- Open the script and click "Run".

3. Using Jupyter Notebook:

- Write code in a cell and press Shift + Enter.

Python QB

4. Using Online Python Interpreters:

- Websites like Replit and Google Colab allow script execution.
-

6) Explain the Structure of a Python Program.

A Python program typically consists of:

1. **Shebang Line (Optional):** `#!/usr/bin/python3` (for Linux)
2. **Module Imports:** `import math`
3. **Function Definitions:** `def my_function():`
4. **Main Code Execution:**

```
python
CopyEdit
if __name__ == "__main__":
    print("Python program structure")
```

7) Explain Commenting in Python Programming.

Comments help in making code readable and understandable.

- **Single-line Comment:**

```
# This is a comment
print("Hello, Python!") # This is also a comment
```

- **Multi-line Comment:**

```
"""
This is a multi-line comment.
Used for documentation.
"""
print("Python supports multi-line comments")
```

8) Explain Input and Output in Python Programming.

- **Input in Python:** Used to take user input using the `input()` function.

```
name = input("Enter your name: ")
print("Hello,", name)
```

- **Output in Python:** Displaying output using the `print()` function.

```
print("Welcome to Python")
```

Python QB

- **Formatted Output:** Using f-strings.

```
age = 25
print(f"I am {age} years old")
```

You said:

Unit No.2 • Question at Remember Level (2 Marks) 1) List data types of Python. 2) Define the terms i)Loop ii)Program. iii)Operator iv)Control Flow 3) List different conditional statements in python. 4) List different loops available in python. 5) State use of break statement in a loop. 6) State use of Continue statement 7) Write a short note on Data Types of python. 8) Write a short note on Membership Operators in Python. 9) Write a short note on Identity Operators in Python.

ChatGPT said:

1) List Data Types of Python.

Python provides various built-in data types, including:

- **Numeric Types:** int, float, complex
 - **Sequence Types:** str, list, tuple, range
 - **Set Types:** set, frozenset
 - **Mapping Type:** dict
 - **Boolean Type:** bool
 - **Binary Types:** bytes, bytearray, memoryview
-

2) Define the Terms:

- **i) Loop:** A loop is a control structure that allows repeating a block of code multiple times until a condition is met.
 - **ii) Program:** A program is a set of instructions written in a programming language to perform a specific task.
 - **iii) Operator:** An operator is a symbol that performs an operation on one or more operands, such as +, -, *, /.
 - **iv) Control Flow:** Control flow refers to the order in which individual statements, instructions, or function calls are executed in a program.
-

3) List Different Conditional Statements in Python.

Python provides the following conditional statements:

- if statement
- if-else statement
- if-elif-else statement
- Nested if statement

Python QB

Example:

```
age = 18
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

4) List Different Loops Available in Python.

Python supports two main types of loops:

- **for loop** (Iterates over a sequence)
- **while loop** (Runs until a condition is false)

Example of for loop:

```
for i in range(5):
    print(i)
```

Example of while loop:

```
x = 0
while x < 5:
    print(x)
    x += 1
```

5) State Use of break Statement in a Loop.

The break statement is used to exit a loop prematurely when a specific condition is met.

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```

6) State Use of `continue` Statement.

The `continue` statement is used to skip the current iteration and move to the next one.

Example:

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output:

```
0  
1  
3  
4
```

(Note: 2 is skipped.)

7) Write a Short Note on Data Types of Python.

Python has several built-in data types categorized as:

- **Numeric Types:** `int`, `float`, `complex` (for numbers)
- **Sequence Types:** `str` (for text), `list`, `tuple` (for ordered collections)
- **Mapping Type:** `dict` (for key-value pairs)
- **Set Types:** `set`, `frozenset` (for unique elements)
- **Boolean Type:** `bool` (`True` or `False`)

Example:

```
x = 10      # int  
y = 3.14    # float  
z = "Python" # str  
a = [1, 2, 3] # list
```

8) Write a Short Note on Membership Operators in Python.

Membership operators are used to check whether a value exists in a sequence (such as a list, tuple, or string).

- `in` → Returns `True` if the value exists in the sequence.
- `not in` → Returns `True` if the value does not exist in the sequence.

Python QB

Example:

```
nums = [1, 2, 3, 4, 5]
print(3 in nums)    # True
print(6 not in nums) # True
```

9) Write a Short Note on Identity Operators in Python.

Identity operators compare the memory locations of two objects.

- `is` → Returns True if both variables refer to the same object.
- `is not` → Returns True if they refer to different objects.

Example:

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b) # True (same object)
print(a is c) # False (different objects with same values)
print(a is not c) # True
40
```

You said:

In depth with ex1) Explain Identity operators in python. 2) Explain Membership operators in python. 3) Explain Decision making statements of python. 4) Explain for loop of python with syntax. 5) Explain while loop of python

ChatGPT said:

1) Explain Identity Operators in Python.

Identity operators are used to compare the memory location of two objects, checking whether they refer to the same object.

Types of Identity Operators:

1. **is Operator:** Returns True if both variables point to the same object.
2. **is not Operator:** Returns True if both variables point to different objects.

Example:

```
a = [1, 2, 3]
b = a # 'b' refers to the same list as 'a'
c = [1, 2, 3] # 'c' is a new list with the same values
```

Python QB

```
print(a is b) # True (Both refer to the same object)
print(a is c) # False (Different objects with same values)
print(a is not c) # True (They are different objects)
```

Explanation:

- `a is b` is **True** because both refer to the same object.
 - `a is c` is **False** because `c` is a different object, even though it has the same values.
 - `a is not c` is **True** because they are different objects in memory.
-

2) Explain Membership Operators in Python.

Membership operators check if a value is present in a sequence (list, tuple, string, dictionary, etc.).

Types of Membership Operators:

1. **in Operator:** Returns True if the value exists in the sequence.
2. **not in Operator:** Returns True if the value does not exist in the sequence.

Example:

```
numbers = [10, 20, 30, 40]
print(20 in numbers) # True (20 is in the list)
print(50 not in numbers) # True (50 is not in the list)
```

Example with Strings:

```
text = "Python Programming"
print("Python" in text) # True
print("Java" not in text) # True
```

Example with Dictionary Keys:

```
person = {"name": "Alice", "age": 25}
print("name" in person) # True (Key exists)
print("height" not in person) # True (Key doesn't exist)
```

3) Explain Decision-Making Statements in Python.

Decision-making statements in Python control the flow of execution based on conditions.

Types of Decision-Making Statements:

Python QB

1. **if Statement**
2. **if-else Statement**
3. **if-elif-else Statement**
4. **Nested if Statement**

1) if Statement

Executes a block of code if the condition is True.

```
age = 18
if age >= 18:
    print("You are an adult")
```

Output:

You are an adult

2) if-else Statement

Executes one block if True, otherwise executes another block.

```
num = 10
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

3) if-elif-else Statement

Used when multiple conditions need to be checked.

```
marks = 85
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
else:
    print("Grade: C")
```

4) Nested if Statement

An if statement inside another if.

```
num = 5
if num > 0:
    if num % 2 == 0:
        print("Even number")
    else:
        print("Odd number")
```

Python QB

4) Explain for Loop in Python with Syntax.

A for loop is used to iterate over a sequence (list, tuple, string, range, etc.).

Syntax:

```
for variable in sequence:  
    # Code to execute
```

Example 1: Iterating over a List

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output:

```
apple  
banana  
cherry
```

Example 2: Using range() Function

```
for i in range(1, 6): # Loops from 1 to 5  
    print(i)
```

Output:

```
1  
2  
3  
4  
5
```

Example 3: Looping through a String

```
word = "Python"  
for letter in word:  
    print(letter)
```

Example 4: Using break and continue in for Loop

```
for i in range(5):  
    if i == 3:  
        break # Stops loop at 3  
    print(i)
```

Output:

```
0
```

Python QB

1
2

5) Explain while Loop in Python.

A while loop is used when a block of code needs to execute repeatedly until a condition is False.

Syntax:

```
while condition:  
    # Code to execute
```

Example 1: Basic while Loop

```
x = 1  
while x <= 5:  
    print(x)  
    x += 1
```

Output:

1
2
3
4
5

Example 2: Using break in while Loop

```
x = 1  
while x <= 10:  
    if x == 5:  
        break # Stops loop at 5  
    print(x)  
    x += 1
```

Output:

1
2
3
4

Example 3: Using continue in while Loop

```
x = 0  
while x < 5:  
    x += 1  
    if x == 3:  
        continue
```

Python QB

```
        continue # Skips printing 3
    print(x)
```

Output:

```
1
2
4
5
```

Example 4: Infinite Loop (Avoid this!)

```
while True:
    print("This will run forever!")
```

(This loop runs indefinitely unless manually stopped.)

You said:

1) Write a Python Program to find the square root of a number. 2) Write a Python Program to swap the values of two variables. 3) Write a Python Program to find out absolute value of an input number. 4) Write a Python Program to find whether a number is even or odd. 5) Write a Python Program to check if the input year is a leap year or not. 6) Write a Python Program to check the largest number among the three numbers. 7) Write a Python Program to display Fibonacci Series. 8) Write a Python Program to find out the reverse of the given number. 9) Write a Python Program to find sum of digit of a number. 10) Write a Python Program to check whether the input number is Armstrong. 11) Write a Python Program to print prime numbers in between a range. 12) Write a Python Program to check whether the entered number is prime or not.

ChatGPT said:

1) Python Program to Find the Square Root of a Number

```
num = float(input("Enter a number: "))
sqrt = num ** 0.5
print(f"The square root of {num} is {sqrt}")
```

Output:

```
Enter a number: 25
The square root of 25.0 is 5.0
```

2) Python Program to Swap Two Variables

```
a = int(input("Enter first number: "))
```

Python QB

```
b = int(input("Enter second number: "))

a, b = b, a # Swapping values

print(f"After swapping: a = {a}, b = {b}")
```

3) Python Program to Find Absolute Value of a Number

```
num = float(input("Enter a number: "))
absolute_value = abs(num)
print(f"The absolute value of {num} is {absolute_value}")
```

4) Python Program to Check Whether a Number is Even or Odd

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

5) Python Program to Check if a Year is a Leap Year

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a leap year")
else:
    print(f"{year} is not a leap year")
```

6) Python Program to Find the Largest of Three Numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

largest = max(a, b, c)
print(f"The largest number is {largest}")
```

7) Python Program to Display Fibonacci Series

```
n = int(input("Enter the number of terms: "))
a, b = 0, 1
print("Fibonacci Series:", a, b, end=" ")

for _ in range(n - 2):
    c = a + b
```

Python QB

```
print(c, end=" ")  
a, b = b, c
```

8) Python Program to Find Reverse of a Number

```
num = int(input("Enter a number: "))  
reverse = int(str(num)[::-1])  
print(f"Reverse of {num} is {reverse}")
```

9) Python Program to Find Sum of Digits of a Number

```
num = int(input("Enter a number: "))  
sum_of_digits = sum(int(digit) for digit in str(num))  
print(f"Sum of digits of {num} is {sum_of_digits}")
```

10) Python Program to Check if a Number is Armstrong

```
num = int(input("Enter a number: "))  
sum_of_cubes = sum(int(digit) ** 3 for digit in str(num))  
  
if num == sum_of_cubes:  
    print(f"{num} is an Armstrong number")  
else:  
    print(f"{num} is not an Armstrong number")
```

11) Python Program to Print Prime Numbers in a Given Range

```
lower = int(input("Enter lower range: "))  
upper = int(input("Enter upper range: "))  
  
print(f"Prime numbers between {lower} and {upper}:")  
for num in range(lower, upper + 1):  
    if num > 1:  
        for i in range(2, int(num ** 0.5) + 1):  
            if num % i == 0:  
                break  
        else:  
            print(num, end=" ")
```

12) Python Program to Check Whether a Number is Prime

```
num = int(input("Enter a number: "))  
if num > 1:  
    for i in range(2, int(num ** 0.5) + 1):  
        if num % i == 0:  
            print(f"{num} is not a prime number")
```

Python QB

```
        break
    else:
        print(f"{num} is a prime number")
else:
    print(f"{num} is not a prime number")
```

You said:

13) Write a Python Program to print and sum of all even numbers between 1 to 20. 14) Write a Python Program to find factorial of a given numbers. 15) Write a python program to print multiplication table of given number. 16) Write a Python Program to find the best of two test average marks out of three test's marks accepted from the user. 17) Write a Python Program to print patterns of * using for loop.

ChatGPT said:

13) Python Program to Print and Sum of All Even Numbers Between 1 to 20

```
sum_even = 0
print("Even numbers between 1 and 20:")
for num in range(2, 21, 2): # Looping through even numbers
    print(num, end=" ")
    sum_even += num
print(f"\nSum of even numbers: {sum_even}")
```

Output:

```
Even numbers between 1 and 20:
2 4 6 8 10 12 14 16 18 20
Sum of even numbers: 110
```

14) Python Program to Find Factorial of a Given Number

```
num = int(input("Enter a number: "))
factorial = 1

if num < 0:
    print("Factorial does not exist for negative numbers")
elif num == 0:
    print("Factorial of 0 is 1")
else:
    for i in range(1, num + 1):
        factorial *= i
    print(f"Factorial of {num} is {factorial}")
```

Example Output:

```
Enter a number: 5
```

Python QB

Factorial of 5 is 120

15) Python Program to Print Multiplication Table of Given Number

```
num = int(input("Enter a number: "))
print(f"Multiplication Table of {num}:")
for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

Example Output:

```
Enter a number: 7
Multiplication Table of 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

16) Python Program to Find the Best of Two Test Average Marks Out of Three Test Marks

```
test1 = float(input("Enter marks for Test 1: "))
test2 = float(input("Enter marks for Test 2: "))
test3 = float(input("Enter marks for Test 3: "))

best_two_tests = sorted([test1, test2, test3], reverse=True)[:2] # Select top 2 marks
average = sum(best_two_tests) / 2

print(f"Best of two test average: {average}")
```

Example Output:

```
Enter marks for Test 1: 85
Enter marks for Test 2: 90
Enter marks for Test 3: 80
Best of two test average: 87.5
```

17) Python Program to Print Patterns of * Using For Loop

Pattern 1: Right-Angled Triangle

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i)
```

Python QB

Example Output for rows = 5:

```
*
**
***
****
*****
```

Pattern 2: Inverted Right-Angled Triangle

```
rows = int(input("Enter number of rows: "))
for i in range(rows, 0, -1):
    print("*" * i)
```

Example Output:

```
*****
****
***
**
*
```

Pattern 3: Pyramid

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

Example Output:

```

 *
***
*****
*****
*****
```

1) Define List, Tuples, Sets and Dictionaries in Python . 2) How to define and access the elements of lists. 3) How to define and access the elements of tuples. 4) State Mutable and Immutable data types in python. 5) How to create directory in python. 6) How to create sets in python. ● Question at Understanding Level. (4 marks) 1) Compare list and Tuple. 2) Explain Union, Intersection, Difference and symmetric Difference operation of sets in python . 3) How to delete an elements/item from dictionary. 4) How to delete an elements/item from tuple. 5) Differentiate between list and dictionaries. 6) Explain Directory methods in python. 7) Explain append, extend, insert method of list in python. 8) Explain properties of dictionary keys 9) How to delete an elements/item from sets

1) Define List, Tuples, Sets, and Dictionaries in Python.

- **List:** A list is an ordered, mutable collection that can store heterogeneous elements. It is defined using square brackets [].

Python QB

```
my_list = [1, "hello", 3.5, True]
```

- **Tuple:** A tuple is an ordered, immutable collection that can store heterogeneous elements. It is defined using parentheses ().

```
my_tuple = (1, "hello", 3.5, True)
```

- **Set:** A set is an unordered collection of unique elements. It is defined using curly braces {} or the set() function.

```
my_set = {1, 2, 3, 4}
```

- **Dictionary:** A dictionary is an unordered collection of key-value pairs. It is defined using curly braces {} with keys and values.

```
my_dict = {"name": "Alice", "age": 25}
```

2) How to Define and Access Elements of a List

Define a List:

```
my_list = [10, 20, 30, 40, 50]
```

Access Elements:

```
print(my_list[0]) # Access first element → 10  
print(my_list[-1]) # Access last element → 50  
print(my_list[1:4]) # Slicing → [20, 30, 40]
```

3) How to Define and Access Elements of a Tuple

Define a Tuple:

```
my_tuple = (10, 20, 30, 40, 50)
```

Access Elements:

```
print(my_tuple[0]) # First element → 10  
print(my_tuple[-1]) # Last element → 50  
print(my_tuple[1:4]) # Slicing → (20, 30, 40)
```

4) Mutable and Immutable Data Types in Python

- **Mutable Data Types:** Can be modified after creation. Examples:
 - List: `my_list[0] = 100`

Python QB

- Dictionary: `my_dict["name"] = "Bob"`
 - Sets: `my_set.add(10)`
 - **Immutable Data Types:** Cannot be modified after creation. Examples:
 - Tuple: `my_tuple[0] = 100` (Error)
 - String: `my_string[0] = "H"` (Error)
 - Numbers (int, float)
-

5) How to Create a Directory in Python

```
import os
os.mkdir("my_directory") # Creates a directory named 'my_directory'
```

Using pathlib:

```
from pathlib import Path
Path("my_directory").mkdir()
```

6) How to Create Sets in Python

```
my_set = {1, 2, 3, 4} # Using curly braces
empty_set = set() # Empty set (cannot use `{}` as it creates a dictionary)
```

Understanding-Level Questions (4 Marks)

1) Compare List and Tuple

Feature	List	Tuple
Mutable	Yes	No
Ordered	Yes	Yes
Indexing	Yes	Yes
Performance	Slower	Faster
Syntax	[]	()

2) Set Operations (Union, Intersection, Difference, Symmetric Difference)

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
print(A | B) # Union → {1, 2, 3, 4, 5, 6}
print(A & B) # Intersection → {3, 4}
```

Python QB

```
print(A - B) # Difference → {1, 2}
print(A ^ B) # Symmetric Difference → {1, 2, 5, 6}
```

3) How to Delete an Item from a Dictionary

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
del my_dict["age"] # Removes 'age' key
print(my_dict) # {'name': 'Alice', 'city': 'New York'}
```

Using pop():

```
my_dict.pop("city")
```

4) How to Delete an Item from a Tuple

Tuples are **immutable**, so we cannot delete elements directly. Instead, we can **convert to a list**, delete an element, and convert back.

```
my_tuple = (10, 20, 30, 40)
temp_list = list(my_tuple)
temp_list.remove(20)
my_tuple = tuple(temp_list)
print(my_tuple) # (10, 30, 40)
```

5) Difference Between List and Dictionary

Feature	List	Dictionary
Data Storage	Stores elements in order	Stores key-value pairs
Mutable?	Yes	Yes
Indexing	Yes (0-based)	No (Keys used)
Access Speed	Slower	Faster (Key-based lookup)

6) Explain Directory Methods in Python

- `os.mkdir("dir_name")` → Creates a directory
 - `os.rmdir("dir_name")` → Deletes a directory
 - `os.listdir("dir_name")` → Lists contents of a directory
 - `os.getcwd()` → Gets current working directory
-

7) Explain `append()`, `extend()`, and `insert()` Methods of List

Python QB

```
my_list = [1, 2, 3]
my_list.append(4) # Adds a single element → [1, 2, 3, 4]

my_list.extend([5, 6]) # Adds multiple elements → [1, 2, 3, 4, 5, 6]
my_list.insert(1, 10) # Inserts 10 at index 1 → [1, 10, 2, 3, 4, 5, 6]
```

8) Properties of Dictionary Keys

1. **Keys must be unique** → Duplicate keys overwrite values.
2. **Keys must be immutable** → Integers, strings, tuples are valid; lists are not.
3. **Keys are case-sensitive** → "Name" and "name" are different.

```
my_dict = {1: "One", "two": 2, (3, 4): "Tuple Key"}
```

9) How to Delete an Element from a Set

```
my_set = {1, 2, 3, 4, 5}
my_set.remove(3) # Removes 3, raises error if not found
my_set.discard(2) # Removes 2, no error if not found
my_set.pop() # Removes and returns a random element
my_set.clear() # Removes all elements
print(my_set) # {}
```

Let me know if you need further clarifications!

Question at Application Level. (4 marks) 1) Explain append, extend, insert method of list in python with example. 2) Explain Dictionary membership test with example. 3) Explain how to delete an item from tuple and how to delete an entire tuple with example. 5) Explain How to delete an elements/item from dictionary with example. 6) Explain How to delete an elements/item from sets with example. 7) Explain Union, Intersection, Difference and symmetric Difference operation of sets in python with example.

ChatGPT said:

Application Level Questions (4 Marks Each)

1) append(), extend(), and insert() Methods of List with Examples

append() Method:

- Adds a single element to the end of the list.
- Syntax: list.append(element)

```
my_list = [1, 2, 3]
my_list.append(4)
```

Python QB

```
print(my_list) # Output: [1, 2, 3, 4]
```

extend() Method:

- Adds multiple elements (from another iterable) to the list.
- Syntax: `list.extend(iterable)`

```
my_list = [1, 2, 3]
my_list.extend([4, 5, 6])
print(my_list) # Output: [1, 2, 3, 4, 5, 6]
```

insert() Method:

- Inserts an element at a specified index.
- Syntax: `list.insert(index, element)`

```
my_list = [1, 2, 4, 5]
my_list.insert(2, 3) # Insert 3 at index 2
print(my_list) # Output: [1, 2, 3, 4, 5]
```

2) Dictionary Membership Test with Example

- The `in` and `not in` operators check if a key exists in a dictionary.

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}

print("name" in my_dict) # Output: True
print("country" in my_dict) # Output: False
print("age" not in my_dict) # Output: False
```

3) How to Delete an Item from a Tuple and Delete an Entire Tuple

Deleting an Item from a Tuple (Indirect Method, as Tuples are Immutable)

Since tuples are immutable, we cannot delete elements directly. Instead, we convert the tuple to a list, remove the element, and convert it back.

```
my_tuple = (1, 2, 3, 4, 5)
temp_list = list(my_tuple)
temp_list.remove(3) # Remove 3
my_tuple = tuple(temp_list)
print(my_tuple) # Output: (1, 2, 4, 5)
```

Deleting an Entire Tuple

Python QB

We can delete an entire tuple using del.

```
my_tuple = (1, 2, 3, 4, 5)
del my_tuple
# print(my_tuple) # This will give an error as my_tuple is deleted.
```

4) How to Delete an Element from a Dictionary with Example

- **Using del Statement:** Removes an item by key.

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
del my_dict["age"]
print(my_dict) # Output: {'name': 'Alice', 'city': 'New York'}
```

- **Using pop() Method:** Removes an item and returns its value.

```
value = my_dict.pop("city")
print(value) # Output: 'New York'
print(my_dict) # Output: {'name': 'Alice'}
```

- **Using clear() Method:** Removes all elements.

```
my_dict.clear()
print(my_dict) # Output: {}
```

5) How to Delete an Element from a Set with Example

- **Using remove() Method:** Removes an item but raises an error if it doesn't exist.

```
my_set = {1, 2, 3, 4}
my_set.remove(2)
print(my_set) # Output: {1, 3, 4}
```

- **Using discard() Method:** Removes an item but does NOT raise an error if it doesn't exist.

```
my_set.discard(5) # No error even if 5 is not present
print(my_set) # Output: {1, 3, 4}
```

- **Using pop() Method:** Removes and returns a random item.

```
python
CopyEdit
removed_item = my_set.pop()
print(removed_item)
print(my_set) # Remaining elements
```

- **Using clear() Method:** Removes all elements.

Python QB

```
my_set.clear()
print(my_set) # Output: set()
```

6) Set Operations: Union, Intersection, Difference, and Symmetric Difference

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
# Union: Combines both sets
print(A | B) # Output: {1, 2, 3, 4, 5, 6}
```

```
# Intersection: Common elements
print(A & B) # Output: {3, 4}
```

```
# Difference: Elements in A but not in B
print(A - B) # Output: {1, 2}
```

```
# Symmetric Difference: Elements in either A or B but not both
print(A ^ B) # Output: {1, 2, 5, 6}
```

Unit 4:\

:

1) Define Function, Module, and Package in Python:

- **Function:** A function is a block of reusable code that performs a specific task. It is defined using the `def` keyword.
- **Module:** A module is a file containing Python code (functions, classes, or variables) that can be imported and used in other programs.
- **Package:** A package is a collection of related modules organized in directories with an `__init__.py` file to distinguish it from a regular directory.

2) Define Exception:

An **exception** is an unexpected event that occurs during program execution, disrupting the normal flow. It can be handled using `try-except` blocks.

3) Enlist Advantages of User-Defined Functions:

- Code reusability
- Increases modularity
- Reduces redundancy
- Improves readability
- Simplifies debugging

4) Write Syntax of a User-Defined Function:

```
python
CopyEdit
def function_name(parameters):
    # Function body
    return value # (optional)
```

5) Enlist Types of Function Arguments:

- **Positional arguments**
- **Keyword arguments**
- **Default arguments**
- **Variable-length arguments** (`*args` and `**kwargs`)

6) State Use of Namespace in Python:

A **namespace** in Python is a system that assigns unique names to objects, preventing naming conflicts. It helps in managing variable scope.

7) Write Syntax to Import Module in Python:

```
python
CopyEdit
import module_name
# or
from module_name import function_name
```

8) Define the Terms:

- **Error:** A mistake in the program that prevents execution (e.g., syntax errors).
- **Exception:** A runtime error that disrupts execution but can be handled using try-except.

9) List Predefined Exceptions in Python:

- ZeroDivisionError
- TypeError
- ValueError
- IndexError
- KeyError
- NameError
- AttributeError
- FileNotFoundError

10) What is a Recursive Function?

A **recursive function** is a function that calls itself to solve a problem.

Example:

```
python

def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5)) # Output: 120
40
```

You said:

1) Explain types of function arguments. 2) Explain types of namespaces in python. 3) Explain how to handle exception in python programming. 4) Explain try ---except block for exception handling in python. 5) How to create a user defined exception? 6) Differentiate between local variable and global variable. 7) Explain various functions of math module. 8) Explain with program how to create user defined exception in python.

ChatGPT said:

1) Types of Function Arguments in Python

Python provides four types of function arguments:

1. **Positional Arguments:** Arguments passed in the same order as function parameters.

```
python
CopyEdit
def greet(name, age):
    print(f'Hello, {name}! You are {age} years old.')

greet("Alice", 25) # Output: Hello, Alice! You are 25 years old.
```

2. **Keyword Arguments:** Arguments passed with parameter names, allowing any order.

```
python
CopyEdit
greet(age=30, name="Bob") # Output: Hello, Bob! You are 30 years old.
```

3. **Default Arguments:** Arguments with default values, making them optional.

```
python
CopyEdit
def greet(name="Guest"):
    print(f'Hello, {name}!')

greet() # Output: Hello, Guest!
greet("Eve") # Output: Hello, Eve!
```

4. **Variable-Length Arguments (*args and **kwargs):**

- *args allows passing multiple positional arguments as a tuple.
- **kwargs allows passing multiple keyword arguments as a dictionary.

```
python
CopyEdit
def add_numbers(*args):
    return sum(args)

print(add_numbers(1, 2, 3, 4)) # Output: 10

def person_info(**kwargs):
    for key, value in kwargs.items():
        print(f'{key}: {value}')

person_info(name="Alice", age=25, city="New York")
```

2) Types of Namespaces in Python

1. **Built-in Namespace:** Contains predefined names like print(), len(), int(), etc.

Python QB

2. **Global Namespace:** Contains variables declared at the script level, accessible throughout the program.
3. **Local Namespace:** Contains variables defined inside functions, accessible only within that function.
4. **Enclosing Namespace (Non-Local):** Used in nested functions where the outer function's variables are accessible inside inner functions.

Example:

```
python
CopyEdit
x = 10 # Global Namespace

def outer():
    y = 20 # Enclosing Namespace

    def inner():
        z = 30 # Local Namespace
        print(x, y, z) # Accessing all three levels

    inner()

outer()
```

3) How to Handle Exceptions in Python

Exceptions are handled using try-except blocks to prevent program crashes.

Example:

```
python
CopyEdit
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input! Please enter a number.")
except Exception as e:
    print(f"An error occurred: {e}")
```

4) Try-Except Block for Exception Handling

The try-except block is used to catch and handle exceptions.

Syntax:

Python QB

```
python
CopyEdit
try:
    # Code that may raise an exception
except ExceptionType:
    # Handle the exception
finally:
    # (Optional) Code that runs regardless of exceptions
```

Example:

```
python
CopyEdit
try:
    x = 5 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
finally:
    print("This will always execute.")
```

5) How to Create a User-Defined Exception?

User-defined exceptions are created by subclassing the Exception class.

Example:

```
class MyError(Exception):
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)

try:
    raise MyError("Custom error occurred!")
except MyError as e:
    print(e)
```

6) Difference Between Local and Global Variables

Feature	Local Variable	Global Variable
Scope	Inside a function	Outside all functions
Lifetime	Exists only during function execution	Exists throughout program execution
Access	Accessible only inside the function	Accessible anywhere in the program
Example	def func(): x = 10	x = 10 (outside any function)

Example:

```
python
```

Python QB

```
CopyEdit
x = 100 # Global variable

def my_function():
    x = 50 # Local variable
    print("Local x:", x)

my_function()
print("Global x:", x)
```

7) Various Functions of Math Module

The math module provides mathematical functions:

Function	Description
math.sqrt(x)	Square root of x
math.pow(x, y)	x raised to the power y
math.factorial(x)	Factorial of x
math.sin(x), math.cos(x), math.tan(x)	Trigonometric functions
math.log(x)	Natural logarithm of x
math.ceil(x), math.floor(x)	Rounds up/down a number

Example:

```
import math

print(math.sqrt(16)) # Output: 4.0
print(math.factorial(5)) # Output: 120
print(math.log(10)) # Output: 2.302...
```

8) Creating a User-Defined Exception in Python

Example:

```
class AgeError(Exception):

    def __init__(self, message="Age must be 18 or above"):
        self.message = message
        super().__init__(self.message)

def check_age(age):
    if age < 18:
        raise AgeError
    else:
        print("You are eligible!")

try:
```

Python QB

```
age = int(input("Enter your age: "))
check_age(age)
except AgeError as e:
    print(e)
```

This program raises an exception if the user enters an age below 18.

40

You said:

1) Write a python program to check for ZeroDivisionError Exception. 2) Explain try—finally blocks for exception handling in python with example. 3) Explain exception handling with example using try, except, raise keywords. 4) How python handle exception? Explain with program. 5) How to create module in python? Explain with example. 6) Explain how to create user defined package in python with example. 7) Write a python program to calculate factorial of number using recursive function. 8) Write a python program to find reverse of a given number using user defined function

ChatGPT said:

1) Python Program to Check for ZeroDivisionError Exception

```
python
CopyEdit
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Input: 0

Output: Error: Cannot divide by zero!

2) Try-Finally Blocks for Exception Handling

The finally block is used to execute code regardless of whether an exception occurs.

Example:

```
python
CopyEdit
try:
    file = open("test.txt", "w")
    file.write("Hello, Python!")
finally:
    file.close()
```

Python QB

```
print("File closed successfully.")
```

Output:

```
arduino
CopyEdit
File closed successfully.
```

Even if an exception occurs, the finally block ensures that the file is closed.

3) Exception Handling Using try, except, and raise

```
python
CopyEdit
class NegativeNumberError(Exception):
    pass

def check_positive(num):
    if num < 0:
        raise NegativeNumberError("Negative numbers are not allowed!")
    return num

try:
    value = int(input("Enter a number: "))
    print("Positive Number:", check_positive(value))
except NegativeNumberError as e:
    print("Error:", e)
```

Input: -5

Output: Error: Negative numbers are not allowed!

4) How Python Handles Exceptions

Python handles exceptions using try-except blocks to prevent crashes.

Example:

```
python
CopyEdit
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    result = num1 / num2
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
```

Python QB

```
print("Error: Invalid input! Please enter numbers only.")
except Exception as e:
    print("An unexpected error occurred:", e)
```

5) How to Create a Module in Python

A module is a Python file containing functions, classes, or variables.

Step 1: Create a Python file (math_operations.py)

```
python
CopyEdit
# math_operations.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Step 2: Import and Use the Module in Another Program

```
python
CopyEdit
import math_operations

print(math_operations.add(5, 3))    # Output: 8
print(math_operations.subtract(10, 4)) # Output: 6
```

6) How to Create a User-Defined Package

Step 1: Create a Package Directory (mypackage)

Inside mypackage, create an `__init__.py` file (can be empty).

Step 2: Create a Module (math_utils.py) Inside the Package

```
python
CopyEdit
# mypackage/math_utils.py
def square(n):
    return n * n

def cube(n):
    return n * n * n
```

Step 3: Import and Use the Package in a Python Program

```
python
CopyEdit
from mypackage import math_utils
```

Python QB

```
print(math_utils.square(4)) # Output: 16
print(math_utils.cube(3)) # Output: 27
```

7) Python Program to Calculate Factorial Using Recursion

```
python
CopyEdit
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a number: "))
print("Factorial of", num, "is", factorial(num))
```

Input: 5

Output: Factorial of 5 is 120

8) Python Program to Find Reverse of a Given Number

```
def reverse_number(num):
    rev = 0
    while num > 0:
        rev = rev * 10 + num % 10
        num = num // 10
    return rev

number = int(input("Enter a number: "))
print("Reverse of the number is:", reverse_number(number))
```

Input: 1234

Output: Reverse of the number is: 4321

40

You said:

Question at Understanding Level. (4 marks) 1) Explain different modes of opening a file. 2) Explain open() and close () methods for opening and closing a file. 3) Explain any three methods associated with files in python. 4) Explain Write method, Writelines(list) method of file. 5) Explain read([n]), readline([n]) method, readlines() method of file. 6) With neat example differentiate between readline() and readlines() functions in file-handling.

ChatGPT said:

1) Different Modes of Opening a File in Python

Python provides different file access modes when opening a file using the `open()` function:

Mode	Description
"r"	Read mode (default). Opens the file for reading; raises an error if the file does not exist.
"w"	Write mode. Creates a new file or overwrites an existing one.
"a"	Append mode. Opens the file for writing but does not erase existing content. New data is added at the end.
"r+"	Read and write mode. File must exist.
"w+"	Write and read mode. Overwrites the file if it exists.
"a+"	Append and read mode. Creates a new file if it does not exist.
"rb", "wb", "ab"	Same as above but for binary files.

2) `open()` and `close()` Methods

- **`open()`**: Used to open a file. It returns a file object.

```
python
CopyEdit
file = open("example.txt", "r") # Opens file in read mode
```

- **`close()`**: Closes the file to free up system resources.

```
python
CopyEdit
file.close()
```

Example:

```
python
CopyEdit
file = open("test.txt", "w")
file.write("Hello, Python!") # Writing to file
file.close()
```

3) Three Common Methods Associated with Files

1. **`write(string)`** – Writes a string to the file.
2. **`read(n)`** – Reads `n` characters from the file. If `n` is omitted, reads the entire file.
3. **`readline()`** – Reads a single line from the file.

Python QB

Example:

```
python
CopyEdit
file = open("sample.txt", "w")
file.write("Hello\nPython\nWorld")
file.close()

file = open("sample.txt", "r")
print(file.readline()) # Output: Hello
file.close()
```

4) write() and writelines(list) Methods

- **write(string)** – Writes a single string to a file.
- **writelines(list)** – Writes a list of strings to a file.

Example:

```
python
CopyEdit
file = open("test.txt", "w")

file.write("Hello, Python!\n")
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
file.writelines(lines) # Writing multiple lines

file.close()
```

5) read(n), readline(n), and readlines() Methods

- **read(n)**: Reads *n* characters from the file.
- **readline(n)**: Reads up to *n* characters from a line.
- **readlines()**: Reads all lines and returns them as a list.

Example:

```
python
CopyEdit
file = open("test.txt", "r")
print(file.read(5))    # Reads first 5 characters
print(file.readline()) # Reads next line
print(file.readlines()) # Reads remaining lines as a list
file.close()
```

6) Difference Between readline() and readlines()

Python QB

Function	Description	Returns
readline()	Reads a single line from the file	A single string
readlines()	Reads all lines and returns them as a list	A list of strings

Example:

File (data.txt):

```
Python
Programming
Language
python
CopyEdit
file = open("data.txt", "r")

print("Using readline():", file.readline())
print("Using readlines():", file.readlines())

file.close()
```

Output:

```
scss
CopyEdit
Using readline(): Python

Using readlines(): ['Programming\n', 'Language\n']
```

This shows that readline() reads one line at a time, while readlines() reads all lines and stores them in a list.

40

You said:

- Question at Application Level. (4 marks) 1) Write a python program to read contents of first .txt file and write same content in second.txt file. 2) Write a python program to append data to an existing file 'python.py'. Read data to be appended from the use. Then display the content of entire file. 3) Write a python program to read a text file and print number of lines, words and characters. 4) Write a python program to create a simple file and write some content in it. 5) Write a python program to open a file in write mode and append some content at the end of file. 6) Write a python program to open a file in read mode and print number of occurrences of characters 'a'. Question Bank Unit No.6 ●

ChatGPT said:

1) Python Program to Copy Contents of One File to Another

```
python
CopyEdit
```

Python QB

```
# Open the first file in read mode
with open("first.txt", "r") as file1:
    content = file1.read()

# Open the second file in write mode and write content
with open("second.txt", "w") as file2:
    file2.write(content)

print("Content copied successfully!")
```

2) Python Program to Append Data to an Existing File and Display Content

```
python
CopyEdit
file_name = "python.py"

# Taking user input to append
append_data = input("Enter data to append: ")

# Appending data to file
with open(file_name, "a") as file:
    file.write("\n" + append_data)

# Reading and displaying updated content
with open(file_name, "r") as file:
    print("\nUpdated File Content:")
    print(file.read())
```

3) Python Program to Count Number of Lines, Words, and Characters in a File

```
file_name = "sample.txt"

with open(file_name, "r") as file:
    content = file.read()

lines = content.split("\n")
words = content.split()
characters = len(content)

print("Number of lines:", len(lines))
print("Number of words:", len(words))
print("Number of characters:", characters)
```

4) Python Program to Create a File and Write Content

```
python
CopyEdit
file_name = "myfile.txt"

# Writing content to file
```

Python QB

```
with open(file_name, "w") as file:
    file.write("Hello, this is a test file.\nPython is awesome!")

print("File created and content written successfully.")
```

5) Python Program to Open a File in Write Mode and Append Content

```
python
CopyEdit
file_name = "example.txt"

# Writing initial content
with open(file_name, "w") as file:
    file.write("This is the initial content.\n")

# Appending more content
with open(file_name, "a") as file:
    file.write("This is appended content.\n")

print("Content written and appended successfully.")
```

6) Python Program to Count Occurrences of Character 'a' in a File

```
file_name = "data.txt"

with open(file_name, "r") as file:
    content = file.read()

count_a = content.count("a")

print("Number of occurrences of 'a':", count_a)
```

This program reads the file and counts how many times the letter 'a' appears in it.

40

You said:

- Question at Remember Level (2 Marks) 1) Write syntax of defining class in Python. 2) Define class and object in python. 3) List out features of OOP's in Python. 4) State the use of __init__ method.
- Question at Understanding Level. (4 marks) 1) With neat example explain default constructor concept in Python. 2) Describe 'Self' Parameter with example. 3) With suitable example explain inheritance in Python. 4) Explain method overloading and overriding in python. 5. Illustrate the use of method overriding? Explain with example

ChatGPT said:

Questions at Remember Level (2 Marks)

Python QB

1) Syntax of Defining a Class in Python

python

```
class ClassName:
    def __init__(self, parameters): # Constructor (optional)
        self.attribute = value # Instance variable

    def method_name(self):
        # Method implementation
        pass
```

2) Define Class and Object in Python

- **Class:** A blueprint for creating objects. It defines attributes (variables) and methods (functions).
- **Object:** An instance of a class. It holds actual data and can use class methods.

Example:

python

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car1 = Car("Toyota") # Object creation
print(car1.brand) # Output: Toyota
```

3) Features of OOP in Python

1. **Encapsulation** – Data hiding using private variables (__var).
 2. **Inheritance** – Allows a class to inherit properties of another class.
 3. **Polymorphism** – Methods with the same name but different implementations.
 4. **Abstraction** – Hiding implementation details and exposing only necessary parts.
-

4) Use of __init__ Method

- It is a constructor that initializes an object's attributes when an object is created.
- Automatically called when an object is instantiated.

Example:

```
python
CopyEdit
class Student:
```

Python QB

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

```
s1 = Student("Alice", 20)  
print(s1.name, s1.age) # Output: Alice 20
```

Questions at Understanding Level (4 Marks)

1) Default Constructor Concept in Python

A **default constructor** is a constructor that does not take any parameters (except self).

Example:

```
python  
CopyEdit  
class Demo:  
    def __init__(self): # Default constructor  
        print("This is a default constructor")  
  
obj = Demo() # Output: This is a default constructor
```

2) self Parameter in Python

- Represents the instance of the class.
- Used to access class attributes and methods.

Example:

```
python  
CopyEdit  
class Person:  
    def __init__(self, name):  
        self.name = name  
  
    def display(self):  
        print("Name:", self.name)  
  
p = Person("John")  
p.display() # Output: Name: John
```

3) Inheritance in Python

- **Inheritance** allows a class (child) to inherit attributes and methods from another class (parent).

Python QB

Example:

```
python
CopyEdit
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal): # Inheriting Animal class
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak() # Output: Animal speaks
d.bark()  # Output: Dog barks
```

4) Method Overloading and Overriding in Python

1. **Method Overloading:** Defining multiple methods with the same name but different parameters (not natively supported in Python).

Example: (Using default parameters to achieve overloading)

```
python
CopyEdit
class Math:
    def add(self, a, b, c=0):
        return a + b + c

m = Math()
print(m.add(2, 3))    # Output: 5
print(m.add(2, 3, 4)) # Output: 9
```

2. **Method Overriding:** A child class redefines a method from the parent class.

Example:

```
python
CopyEdit
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    def show(self): # Overriding method
        print("Child class")

c = Child()
c.show() # Output: Child class
```

5) Use of Method Overriding with Example

Method overriding allows a child class to provide a specific implementation of a method that is already defined in its parent class.

Example:

```
python
CopyEdit
class Vehicle:
    def move(self):
        print("Vehicle is moving")

class Car(Vehicle):
    def move(self): # Overriding move() method
        print("Car is moving faster")

c = Car()
c.move() # Output: Car is moving faster
```

This shows that method overriding allows modifying the behavior of a parent class method in a subclass.

40

You said:

Question at Application Level. (4 marks) 1) Design a class student with data members; Name, roll number address. Create suitable method for reading and printing students details. 2) Create a parent class named Animals and a child class Herbivorous which will extend the class Animal. In the child class Herbivorous over side the method feed (). Create a object 3) Write a program to create class EMPLOYEE with ID and NAME and display its contents. 4) Write a program to implement the concept of inheritance in python 5. Design a class student with data members: name, roll no, department, mobile no. Create suitable methods for reading and printing student information. 6) Design a class Employee with data members: name, department and salary. Create suitable methods for reading and printing employee information

ChatGPT said:

1) Python Program for Student Class

```
python
CopyEdit
class Student:
    def __init__(self, name, roll_no, address):
        self.name = name
        self.roll_no = roll_no
        self.address = address

    def display(self):
```

Python QB

```
print(f"Name: {self.name}")
print(f"Roll No: {self.roll_no}")
print(f"Address: {self.address}")

# Creating an object
s1 = Student("Alice", 101, "New York")
s1.display()
```

2) Python Program for Parent-Child Class with Overridden Method

```
python
CopyEdit
class Animal:
    def feed(self):
        print("Animals eat food")

class Herbivorous(Animal):
    def feed(self): # Overriding the method
        print("Herbivorous animals eat plants")

# Creating an object
h = Herbivorous()
h.feed()
```

Output:

```
nginx
CopyEdit
Herbivorous animals eat plants
```

3) Python Program for Employee Class

```
python
CopyEdit
class Employee:
    def __init__(self, emp_id, name):
        self.emp_id = emp_id
        self.name = name

    def display(self):
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")

# Creating an object
emp1 = Employee(101, "John Doe")
emp1.display()
```

4) Python Program to Implement Inheritance

Python QB

```
python
CopyEdit
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f'Name: {self.name}, Age: {self.age}')

class Employee(Person): # Inheriting Person class
    def __init__(self, name, age, emp_id):
        super().__init__(name, age)
        self.emp_id = emp_id

    def display(self):
        super().display()
        print(f'Employee ID: {self.emp_id}')

# Creating an object
e1 = Employee("Alice", 25, 102)
e1.display()
```

5) Python Program for Student Class with Additional Data Members

```
python
CopyEdit
class Student:
    def __init__(self, name, roll_no, department, mobile_no):
        self.name = name
        self.roll_no = roll_no
        self.department = department
        self.mobile_no = mobile_no

    def display(self):
        print(f'Name: {self.name}')
        print(f'Roll No: {self.roll_no}')
        print(f'Department: {self.department}')
        print(f'Mobile No: {self.mobile_no}')

# Creating an object
s1 = Student("Bob", 102, "Computer Science", "9876543210")
s1.display()
```

6) Python Program for Employee Class with Additional Data Members

```
python
CopyEdit
class Employee:
    def __init__(self, name, department, salary):
        self.name = name
        self.department = department
```

Python QB

```
self.salary = salary
```

```
def display(self):  
    print(f"Name: {self.name}")  
    print(f"Department: {self.department}")  
    print(f"Salary: {self.salary}")
```

```
# Creating an object  
emp1 = Employee("David", "HR", 50000)  
emp1.display()
```